Name: Ahmed Mohamed Shawky

Title: AI optimizers

1. # Introduction:

An optimizer in ai is the internal mathematical algorithm responsible for updating the models weights and biases, basically the model makes a prediction, and a loss function calculates how incorrect this prediction was compared to ground truth, the loss function creates a higher dimensional mathematical terrain, the goal of training is to find the minimum value of this terrain, where error has its minimum value.

it tells the model which **direction** the parameters should shift its weight to decrease its error

it also tells the model **step size** that it should take or rather how much to change these parameters

in this report we are going to explore the some of the most widely used optmizers

gradient descent
schochastic gradient descent
AdaGrad
RMSProp
ADAM

2. # Optimizer explanation:

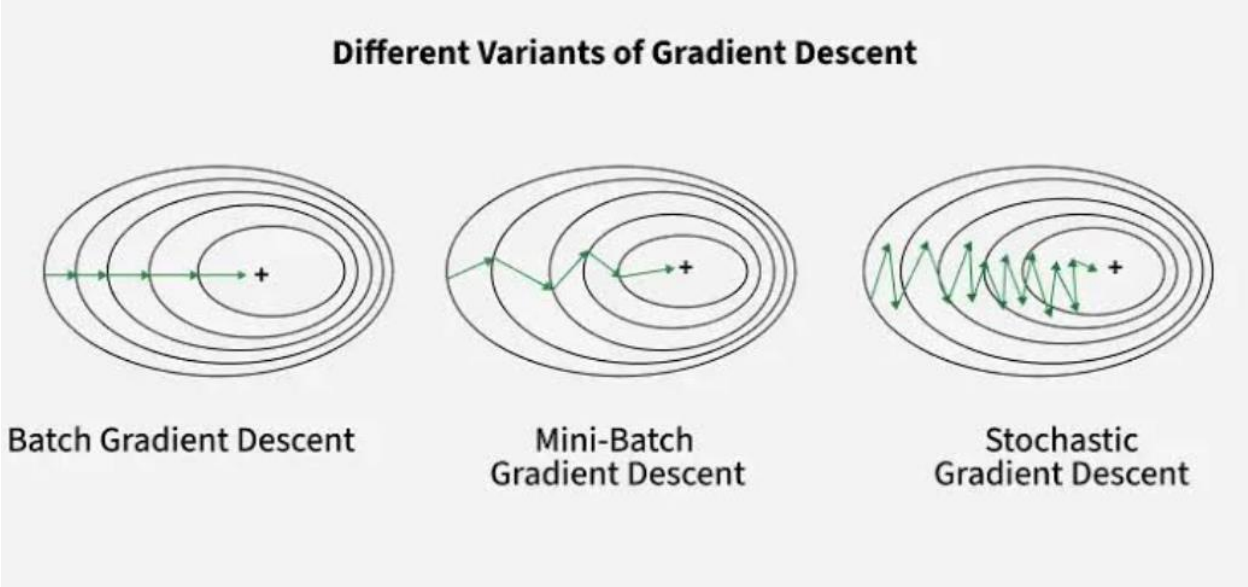
Gradient descent is the fundamental optimization algorithm behind almost all modern machine learning and deep learning models. Its core objective is simple: minimize a loss function by iteratively tweaking the model's parameters in the direction that decreases error the fastest

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta L(\theta_t)$$

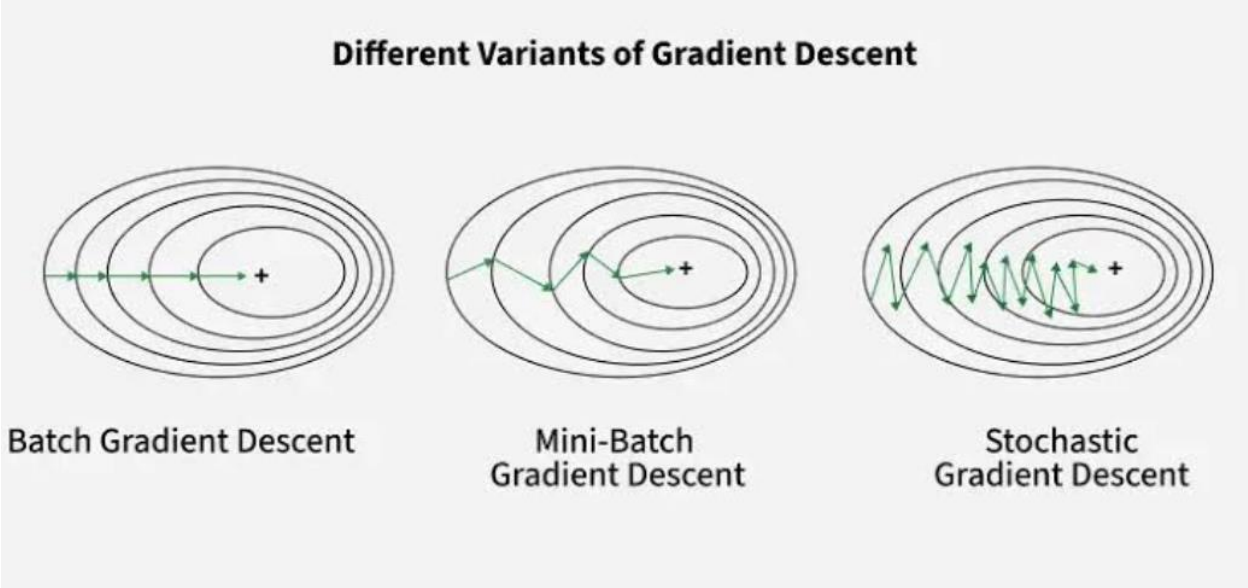
L is the loss function and it basically says how far your model is from ground truth and the small triangle beside it is the gradient, its mathematically defined as the vector of partial derivatives of a loss function with respect to each parameter
η is a hyper parameter called learning rate which we pick its value, it controls the size of the step taken in each iteration
to move towards the minimum we subtract the gradient from the scaled by the learning rate

sadly gradient descent while its a really good optimizer it has its limitations, if the learning rate is too high it might miss the minima completely and diverge, and if its too low it can get stuck or take forever to compute
it also gets trapped in saddle points, local minima, regions where gradient vanishes

Schoastic gradient descent updates the gradient descent by taking only one sample per step (randomly chosen) rather than processing all the data at once



OBJ



Speed & Memory Efficiency: Computing loss for millions of data points per parameter

update is computationally expensive. SGD performs updates instantly, allowing online learning directly on streaming data.

Escaping Local Minima: Because updates depend on individual data points, the gradient estimate contains high variance. This "noise" creates a zigzag trajectory that helps the optimizer bounce out of shallow local minima and bad saddle points

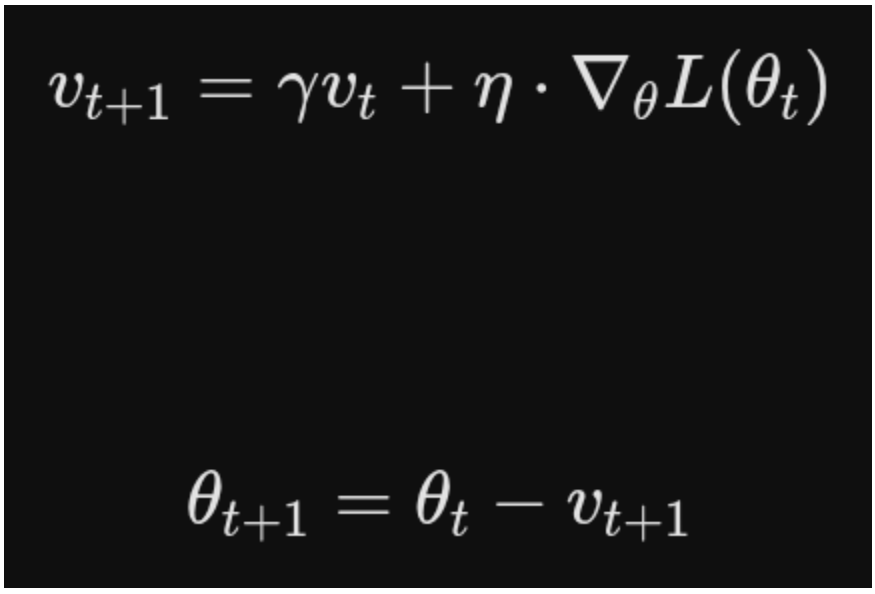
however SGD has some drawbacks

It fails to converge cleanly

sometimes the noise leads to fluctuations which can miss minima and starts oscillating

it fails to utilize GPU tensor cores effectively compared to mini batch or normal gradient, it basically sucks at being parallelly computed, takes a more serial approach which limits it to running on CPUs or special chips just not GPU

a version of SGD uses momentum
Instead of updating parameters purely based on the current gradient, Momentum tracks an Exponentially Weighted Moving Average of past gradients via a velocity vector:



γ Momentum Coefficient: Typically set to 0.9. It controls how much weight is retained from previous updates (the friction factor)
when gradients consistently point in the same direction, velocity grows larger, taking bigger strides. When gradients alternate signs (oscillating back and forth across a ravine), the opposing values cancel each other out in the velocity sum, damping vertical bounces.

it dampens oscillations and accelerates convergence

a really famous version of this momentum SGD is NAG or Nesterov Accelerated Gradient
Standard momentum calculates the gradient at the **current position**. NAG instead projects **where** the current **velocity vector** will carry the parameters first, then computes the gradient **at that look-ahead point**

$$v_{t+1} = \gamma v_t + \eta \cdot \nabla_\theta L(\theta_t - \gamma v_t)$$

This look-ahead mechanism acts as an intelligent brake slowing down before overshooting the target minimum if the gradient starts sloping upward ahead.

GD and SGD with its variants still used 1 learning rate hyperparameter for all parameters in the network, adaptive learning methods changed that by making a learning rate PER parameter based on historical gradients.

AdaGrad (Adaptive Gradient Algorithm)
AdaGrad adjusts the learning rate inversely proportional to the square root of the sum of all historical squared gradients for each parameter.

accumilate square gradients:

$$G_t = G_{t-1} + (\nabla_\theta L(\theta_t))^2$$

parameters updates:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta L(\theta_t)$$

Gt is the running sum of squared gradients for each parameter
epsilon is a tiny number to prevent 0 division

this optimizer showed great results with sparse data like natural language processing, recommendation systems like YouTube, because rarely updated parameters retain a larger effective learning rate.

however because G increases in every step its effective step size shrinks till it reaches 0 where the model just stops learning.

RMSprop (Root Mean Square Propagation)
RMSprop fixes AdaGrad's decaying learning rate flaw by replacing the simple cumulative sum G with an Exponentially Weighted Moving Average (EWMA). This gives recent gradients higher priority while gradually discarding ancient gradient history.

compute moving average for squared gradients:

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla_\theta L(\theta_t))^2$$

then updates the parameters:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \cdot \nabla_\theta L(\theta_t)$$

beta is called decay rate which controls how quickly past gradient history dies

ADAM
widely used, the go to for every deep learning engineer, the “best” optimizer out there, it combines momentum and RMSProp into 1 robust algorithm

first moment vector or momentum:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

estimates the directional trend of the gradient

second moment or RMSProp, estimates the variance of gradient magnititudes

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

parameter update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

Best of Both Worlds, Momentum prevents stalling on flat plateaus; RMSprop scales step size per parameter.

Low Tuning Requirement:
Default hyperparameters, work remarkably well across diverse architectures without heavy adjustment.

Efficient,Computes first-order gradients only, requiring minimal memory overhead while handling noisy or sparse gradients smoothly